What is `PARTITION BY` in SQL?

`PARTITION BY` divides rows into groups based on one or more columns without combining the rows into a single row per group. It is used inside the `OVER()` clause of a window function.

Think of it as: calculate separately for each group, but keep every row.

1. Example: `PARTITION BY department`

Suppose we have this employee table:

Employee	Department	Salary
Amit	IT	50000
Rahul	IT	70000
Priya	IT	60000
Neha	HR	40000
Ravi	HR	50000

Query:

```
SELECT
    employee,
    department,
    salary,
    SUM(salary) OVER (
        PARTITION BY department
    ) AS dept_total
```

FROM employees;

Result:

Employee	Department	Salary	Dept_Total
Amit	IT	50000	180000
Rahul	IT	70000	180000
Priya	IT	60000	180000
Neha	HR	40000	90000
Ravi	HR	50000	90000

How does it work?

IT partition

3 rows

Amit

50,000

Rahul

70,000

Priya

60,000

IT total = 180,000

HR partition

2 rows

Neha

40,000

Ravi

50,000

HR total = 90,000

The database calculates the sum independently for IT and HR, then displays the appropriate total on each employee's row.

2. `PARTITION BY` versus `GROUP BY`

This is the most important distinction.

Using `GROUP BY`:

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department;
```

Result: two rows, one for IT and one for HR.

Using `PARTITION BY`:

```
SELECT
   employee,
   department,
   salary,
   SUM(salary) OVER (PARTITION BY department) AS total_salary
FROM employees;
```

Result: five rows, one for each employee, with the department total repeated.

Feature	`GROUP BY`	`PARTITION BY`

Groups rows	Yes	Yes, for calculation
Preserves individual rows	No, generally	Yes
Common use	Department totals	Department totals alongside employee details

3. Can we use multiple columns in `PARTITION BY`?

Yes. For example:

```
SELECT
    employee,
    department,
    job_title,
    salary,
    SUM(salary) OVER (
        PARTITION BY department, job_title
    ) AS total_salary
FROM employees;
```

Now the calculation is performed separately for each combination of `department` and `job_title`.

For example, IT developers form one partition, IT managers another, and HR developers another.

4. `PARTITION BY` with ranking

Suppose you want to rank employees by salary within each department:

```
SELECT
    employee,
    department,
    salary,
    DENSE_RANK() OVER (
        PARTITION BY department
```

```
      ORDER BY salary DESC
   ) AS salary_rank
FROM employees;
```

Each department starts ranking at 1 independently.

- IT employees are ranked against other IT employees.
- HR employees are ranked against other HR employees.
- `ORDER BY salary DESC` places the highest salary first.

Remember this rule:

- `PARTITION BY` determines which rows belong together.
- `ORDER BY` determines the order of rows within each partition.
- The window function determines what calculation is performed.